Mon Train

Jean Claude Garcia
Formation : TP Développeur Web et Web Mobile
Année 2026

Mon train

1. Présentation du projet

Création d' une application web permettant à un utilisateur de trouver un trajet en train pour aller d'une ville à une autre (qui correspond au trajet des trains à grande vitesse en France).
Cette application est disponible via un site web accessible à tous (jeune, personne âgée, personnes en situation de handicap…). Conscient de l'importance de la navigation sur mobile aujourd'hui, le site fonctionne de manière optimale sur une variété de dispositifs et de tailles d'écran et est conçu en mobile first.
La sécurité étant un point important, il a été mise en place toutes les bonnes pratiques à ce niveau.
Le site a la possibilité de choisir la langue français ou anglais, ainsi que la palette couleur suivant un dégradé avec comme choix : vert, bleu, rouge, jaune, gris.
Afin de faciliter leur développement, les variables de Bootstrap ont été utilisées .
Cette API permet de récupérer les données dynamiquement et d'interroger une base de données MySQL.

2. Analyse des besoins

2.1 Fonctionnalités

Le besoin fonctionnel est relativement simple:
- Créer une application qui s'appuie sur la base de données
- Mettre en place un système d'authentification
- Le MCD du projet au format JPG, PNG ou PDF,
- Le MLD au format textuel,
- Alimenter la base de données
- Réaliser une page d'accueil permettant la connexion
- Créer un dépôt GitHub pour versionner votre travail et permettre le partage du code associé

2.2 Contraintes

Les utilisateurs sont caractérisés par :
• Un nom d'utilisateur (username),
• Un numéro de téléphone (phone),
• Un mot de passe (password).

Les caractéristiques des lignes de train sont :
• Un nom de la ligne de bus (trainLine),

Les caractéristiques des arrets de train sont :
• Un nom de l'arret de train ou ville (cityName),

Les caractéristiques des arrets sont :

Mon train

• Une ligne de train (trainLineId)
• Une ville (cityId)
• Un numéro d'ordre d'arrets (stopOrder)

Les caractéristiques des horaires sont :
• Un arret de train (lineStopId),
• Un horaire (time),

3. MCD – Application de recherche de trajets de trains

- *UTILISATEUR*
  - idUser (PK)
  - username (unique)
  - phone
  - passwordHash
  - role (user | admin)

- *TRAINLINE*
  - idLine (PK)
  - nameLine

- *CITY*
  - idCity (PK)
  - nameCity

- *LINESTOP (association)*

Représente un arrêt dans une ligne avec son ordre

  - idLineStop (PK)
  - trainLineId (FK)
  - cityId (FK)
  - stopOrder

Cardinalités :

Une TRAINLINE possède 1,N LINESTOP

Une CITY apparaît dans 1,N LINESTOP

- ⏱ *SCHEDULE*

Horaire d'un bus à un arrêt précis

- idTime (PK)
- lineStopId (FK)
- time

Cardinalités :

Un LINESTOP possède 0,N SCHEDULE

Un SCHEDULE appartient à 1,1 LINESTOP

Relations (cardinalités)

UTILISATEUR
→ aucune relation métier avec les autres tables (auth seulement)

TRAINLINE (1,N) — contient — LINESTOP (1,1)

CITY (1,N) — est desservie par — LINESTOP (1,1)

LINE_STOP (1,1) — possède — SCHEDULE (0,N)

Relations :

TRAIN_LINE (1,N) ——LINE_STOP —— (N,1) CITY

LINE_STOP (1,1) —— (0,N) SCHEDULE

2. Script SQL

2.1 Création de la base : train_app_script_import_phpmyadmin,sql

```
CREATE DATABASE train_app CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE train_app;

-- ======================
-- UTILISATEURS
-- ======================
CREATE TABLE users (
  idUser INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) NOT NULL UNIQUE,
  phone VARCHAR(20) NOT NULL,
  passwordhash VARCHAR(255) NOT NULL,
  role ENUM('user','admin') DEFAULT 'user',
);

-- ======================
-- LIGNES DE TRAIN
-- ======================
CREATE TABLE trainLines (
  idLine INT AUTO_INCREMENT PRIMARY KEY,
  nameLine VARCHAR(100) NOT NULL,
);

-- ======================
-- VILLES / ARRETS
-- ======================
CREATE TABLE city (
  idCity INT AUTO_INCREMENT PRIMARY KEY,
  nameCity VARCHAR(100) NOT NULL,
);

-- ======================
-- ASSOCIATION LIGNE ↔ ARRET
-- ======================
CREATE TABLE lineStops (
  idLineStop INT AUTO_INCREMENT PRIMARY KEY,
  trainLineId INT NOT NULL,
  cityId INT NOT NULL,
  stopOrder INT NOT NULL,

  CONSTRAINT fk_trainLine
    FOREIGN KEY (trainLineId) REFERENCES trainLines(idLine)
    ON DELETE CASCADE,

  CONSTRAINT fk_city
    FOREIGN KEY (cityId) REFERENCES city(idCity)
```

```
    ON DELETE CASCADE
);

-- ======================
-- HORAIRES
-- ======================
CREATE TABLE schedules (
  idSchedule INT AUTO_INCREMENT PRIMARY KEY,
  lineStopId INT NOT NULL,
  time VARCHAR(5) NOT NULL,

  CONSTRAINT fk_lineStop
    FOREIGN KEY (lineStopId) REFERENCES lineStops(idLinStop)
    ON DELETE CASCADE
);
```

2.2 Jeu de données (seed de test)

```
-- VILLES
INSERT INTO city (nameCity) VALUES
('Paris'),
('Lyon'),
('Marseille'),
('Lille'),
('Bordeaux'),
('Nantes'),
('Strasbourg');

-- LIGNES
INSERT INTO train_lines (nameLine) VALUES
('TGV Nord-Atlantique'),
('TGV Méditerranée');

-- ASSOCIATIONS (ordre des arrêts)
-- TGV Nord-Atlantique : Lille → Paris → Bordeaux
INSERT INTO train_line_stops (stopOrder, trainLineId, cityId) VALUES
(1, 2, 4),
(2, 2, 1),
(3, 2, 5);

-- TGV TGV Méditerranée : Paris → Lyon → Marseille
INSERT INTO train_line_stops (stopOrder, trainLineId, cityId) VALUES
(1, 1, 1),
(2, 1, 2),
(3, 1, 3);
```

```
-- HORAIRES
INSERT INTO schedules (time, lineStopId) VALUES
('08h00', 1),
('10h00', 2),
('12h00', 3);
```

2.3 Base de donnée sur PHPMyAdmin : train_app_script_import_phpmyadmin,sql

```
-- phpMyAdmin SQL Dump
-- version 5.2.1
-- https://www.phpmyadmin.net/
--
-- Hôte : 127.0.0.1
-- Version du serveur : 10.4.32-MariaDB
-- Version de PHP : 8.2.12

SET SQL_MODE = "NO_AUTO_VALUE_ON_ZERO";
START TRANSACTION;
SET time_zone = "+00:00";


/*!40101 SET @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT */;
/*!40101 SET @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS */;
/*!40101 SET @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION */;
/*!40101 SET NAMES utf8mb4 */;

--
-- Base de données : `train_app`
--

-- --------------------------------------------------------

--
-- Structure de la table `city`
--

CREATE TABLE `city` (
  `idCity` int(11) NOT NULL,
  `nameCity` varchar(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Déchargement des données de la table `city`
--

INSERT INTO `city` (`idCity`, `nameCity`) VALUES
```

```
(1, 'Paris'),
(2, 'Lyon'),
(3, 'Marseille'),
(4, 'Lille'),
(5, 'Bordeaux'),
(6, 'Nantes'),
(7, 'Strasbourg');

-- --------------------------------------------------------

--
-- Structure de la table `linestops`
--

CREATE TABLE `linestops` (
  `idLineStop` int(11) NOT NULL,
  `trainLineId` int(11) NOT NULL,
  `cityId` int(11) NOT NULL,
  `stopOrder` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Déchargement des données de la table `linestops`
--

INSERT INTO `linestops` (`idLineStop`, `trainLineId`, `cityId`, `stopOrder`) VALUES
(1, 2, 4, 1),
(2, 2, 1, 2),
(3, 2, 5, 3),
(4, 1, 1, 1),
(5, 1, 2, 2),
(6, 1, 3, 3);

-- --------------------------------------------------------

--
-- Structure de la table `schedules`
--

CREATE TABLE `schedules` (
  `idSchedule` int(11) NOT NULL,
  `lineStopId` int(11) NOT NULL,
  `time` varchar(5) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Déchargement des données de la table `schedules`
--
```

```
INSERT INTO `schedules` (`idSchedule`, `lineStopId`, `time`) VALUES
(1, 1, '08h00'),
(2, 2, '10h00'),
(3, 3, '12h00');

-- --------------------------------------------------------

--
-- Structure de la table `trainlines`
--

CREATE TABLE `trainlines` (
  `idLine` int(11) NOT NULL,
  `nameLine` varchar(100) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Déchargement des données de la table `trainlines`
--

INSERT INTO `trainlines` (`idLine`, `nameLine`) VALUES
(1, 'TGV Nord-Atlantique'),
(2, 'TGV Méditerranée');

-- --------------------------------------------------------

--
-- Structure de la table `users`
--

CREATE TABLE `users` (
  `idUser` int(11) NOT NULL,
  `username` varchar(50) NOT NULL,
  `phone` varchar(20) NOT NULL,
  `passwordhash` varchar(255) NOT NULL,
  `role` enum('user','admin') DEFAULT 'user'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

--
-- Index pour les tables déchargées
--

--
-- Index pour la table `city`
--
ALTER TABLE `city`
  ADD PRIMARY KEY (`idCity`);
```

```
--
-- Index pour la table `linestops`
--
ALTER TABLE `linestops`
  ADD PRIMARY KEY (`idLineStop`);

--
-- Index pour la table `schedules`
--
ALTER TABLE `schedules`
  ADD PRIMARY KEY (`idSchedule`);

--
-- Index pour la table `trainlines`
--
ALTER TABLE `trainlines`
  ADD PRIMARY KEY (`idLine`);

--
-- Index pour la table `users`
--
ALTER TABLE `users`
  ADD PRIMARY KEY (`idUser`),
  ADD UNIQUE KEY `username` (`username`);

--
-- AUTO_INCREMENT pour les tables déchargées
--

--
-- AUTO_INCREMENT pour la table `city`
--
ALTER TABLE `city`
  MODIFY `idCity` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=8;

--
-- AUTO_INCREMENT pour la table `linestops`
--
ALTER TABLE `linestops`
  MODIFY `idLineStop` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=7;

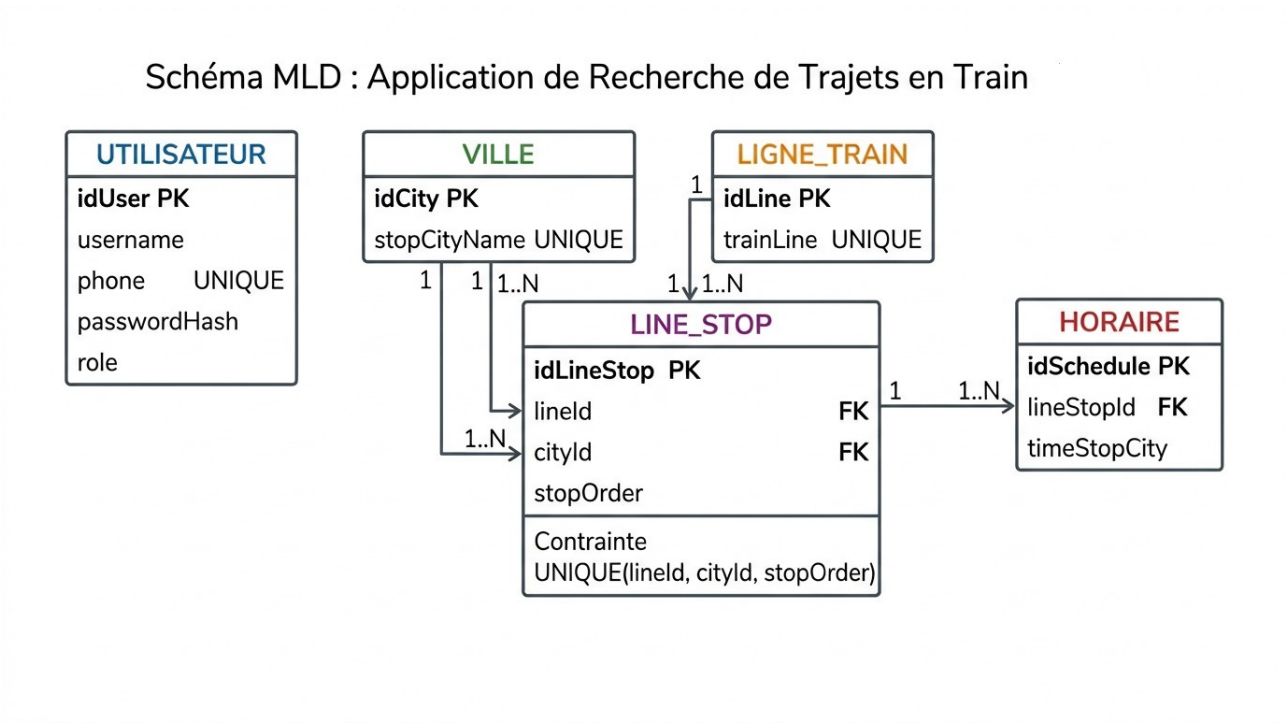
--
-- AUTO_INCREMENT pour la table `schedules`
--
ALTER TABLE `schedules`
  MODIFY `idSchedule` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=4;
```

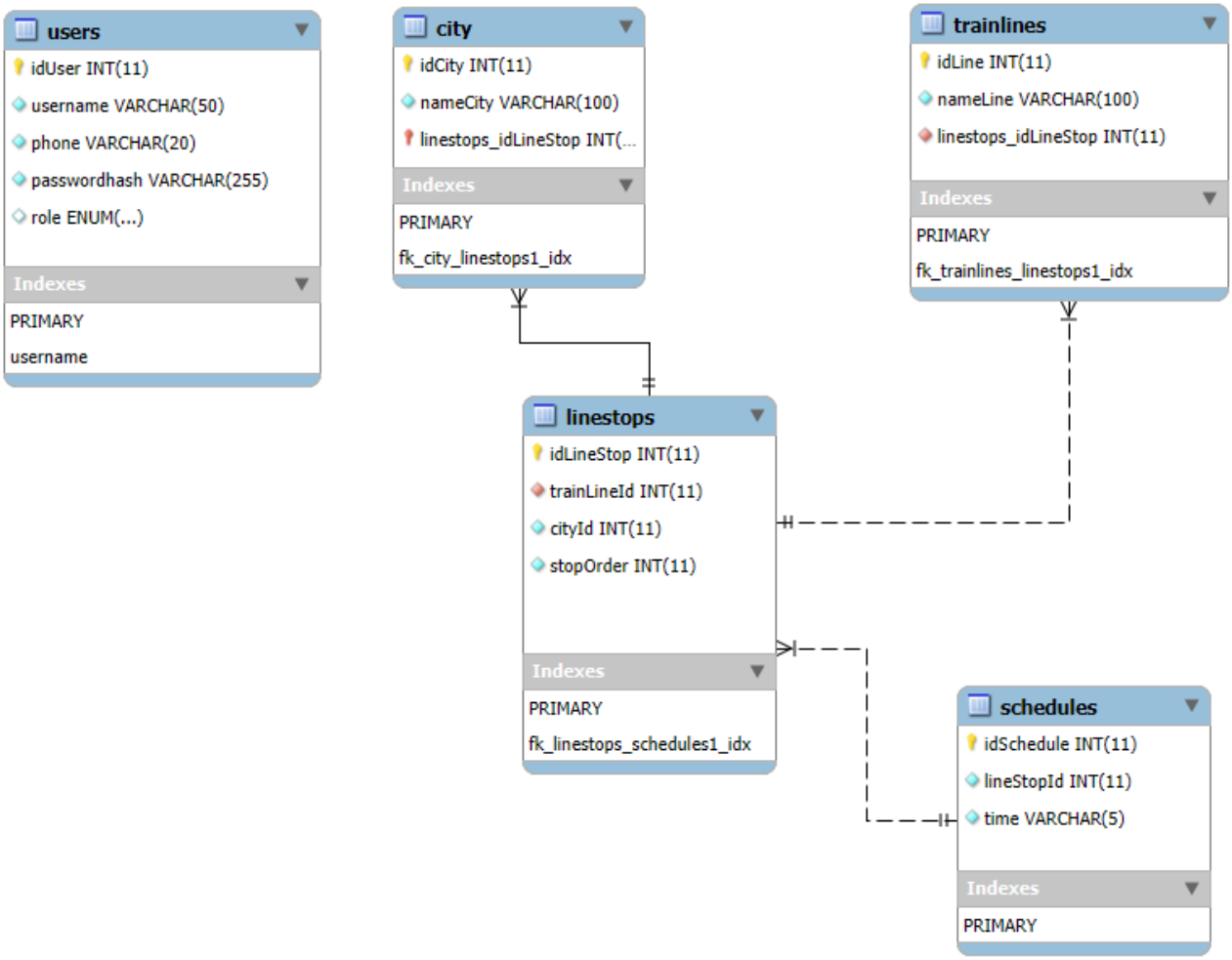
```
--
-- AUTO_INCREMENT pour la table `trainlines`
--
ALTER TABLE `trainlines`
  MODIFY `idLine` int(11) NOT NULL AUTO_INCREMENT, AUTO_INCREMENT=3;

--
-- AUTO_INCREMENT pour la table `users`
--
ALTER TABLE `users`
  MODIFY `idUser` int(11) NOT NULL AUTO_INCREMENT;
COMMIT;

/*!40101 SET CHARACTER_SET_CLIENT=@OLD_CHARACTER_SET_CLIENT */;
/*!40101 SET CHARACTER_SET_RESULTS=@OLD_CHARACTER_SET_RESULTS */;
/*!40101 SET COLLATION_CONNECTION=@OLD_COLLATION_CONNECTION */;
```

2.4 MLD





3. Backend

3.1 Arborescence backend

```
backend/
│── config/
│   └── database.js
│
│── models/
│   ├── index.js
│   ├── User.js
│   ├── TrainLine.js
│   ├── City.js
│   ├── LineStop.js
│   └── Schedule.js
│
```

Mon train

```
│── controllers/
│   ├── authController.js
│   ├── cityController.js
│   ├── searchController.js
│   └── lineController.js
│
│── routes/
│   ├── authRoutes.js
│   ├── lineRoutes.js
│   ├── cityRoutes.js
│   └── searchRoutes.js
│
│── middlewares/
│   └── auth.js
│
│── server.js
│── app.js
│── .env
│── package.json
```

3.2 Installation des dépendances

npm init -y
npm install express sequelize mysql2 bcrypt jsonwebtoken dotenv cors helmet express-rate-limit
npm install nodemon --save-dev

3.3 Fichiers

package.json

```
{
 "name": "my-train-backend",
 "version": "1.0.0",
 "private": true,
 "dependencies": {
  "@testing-library/dom": "^10.4.1",
  "@testing-library/jest-dom": "^6.9.1",
  "@testing-library/react": "^16.3.2",
  "@testing-library/user-event": "^13.5.0",
  "bcrypt": "^6.0.0",
```

```
    "cors": "^2.8.6",
    "dotenv": "^17.3.1",
    "express": "^5.2.1",
    "helmet": "^8.1.0",
    "jsonwebtoken": "^9.0.3",
    "mongoose": "^9.3.0",
    "mysql2": "^3.20.0",
    "react": "^19.2.4",
    "react-dom": "^19.2.4",
    "react-scripts": "5.0.1",
    "sequelize": "^6.37.8",
    "web-vitals": "^2.1.4"
  },
  "scripts": {
    "dev": "nodemon src/server.js",
    "start": "node src/server.js"
  },
  "eslintConfig": {
    "extends": [
      "react-app",
      "react-app/jest"
    ]
  },
  "browserslist": {
    "production": [
      ">0.2%",
      "not dead",
      "not op_mini all"
    ],
    "development": [
      "last 1 chrome version",
      "last 1 firefox version",
      "last 1 safari version"
    ]
  },
  "description": "This project was bootstrapped with [Create React App]
(https://github.com/facebook/create-react-app).",
  "main": "index.js",
  "devDependencies": {
    "jest": "^27.5.1",
    "nodemon": "^3.1.14"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

Mon train

.env

```
APP_NAME=My_Train_App
API_URL=127.0.0.1
PORT=5000
MONGODB_URI=mongodb://localhost:27017/Api_Train

DB_HOST=localhost
DB_PORT=3306
DB_NAME=train_app
DB_USER=root
DB_PASSWORD=jcg3773

JWT_SECRET=123456789
JWT_EXPIRES_IN=100d

FRONT_URL=http://localhost:3000
```

config/database.js

```
const { Sequelize } = require('sequelize');
const mongoose = require('mongoose');
require('dotenv').config();

const sequelize = new Sequelize(
  process.env.DB_NAME,
  process.env.DB_USER,
  process.env.DB_PASSWORD,
  {
    host: process.env.DB_HOST,
    port: process.env.DB_PORT || 3306,
    dialect: 'mysql',
    logging: false,
  }
);

const MONGODB_URI =
  process.env.MONGODB_URI || 'mongodb://localhost:27017/Api_Train';

const connectMongo = async () => {
  try {
    await mongoose.connect(MONGODB_URI, {
      useNewUrlParser: true,
      useUnifiedTopology: true,
    });
```

```
    console.log('Connected to MongoDB');
  } catch (err) {
    console.error('MongoDB connection error', err);
    process.exit(1);
  }
};

module.exports = {
  sequelize,
  connectMongo,
  mongoose,
};
```

Modèles Sequelize

models/User.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const User = sequelize.define(
  'User',
  {
    idUser: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    username: {
      type: DataTypes.STRING(100),
      allowNull: false,
    },
    phone: {
      type: DataTypes.STRING(20),
      allowNull: false,
      unique: true,
    },
    passwordHash: {
      type: DataTypes.STRING(255),
      allowNull: false,
    },
    role: {
      type: DataTypes.ENUM('USER', 'ADMIN'),
      allowNull: false,
      defaultValue: 'USER',
    },
  },
```

```
  {
    tableName: 'utilisateur',
    timestamps: false,
  }
);

module.exports = User;
```

models/TrainLine.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const TrainLine = sequelize.define('TrainLine', {
  name: {
    type: DataTypes.STRING,
    allowNull: false,
  },
});

module.exports = TrainLine;
```

models/City.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const StopCity = sequelize.define(
  'StopCity',
  {
    idCity: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    cityName: {
      type: DataTypes.STRING(100),
      allowNull: false,
      unique: true,
    },
  },
  {
    tableName: 'ville',
    timestamps: false,
  }
);
module.exports = StopCity;
```

models/TrainLine.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const LineStop = sequelize.define(
  'LineStop',
  {
    idLineStop: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
    },
    trainLineId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    stopCityId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    stopOrder: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
  },
  {
    tableName: 'line_stop',
    timestamps: false,
  }
);

module.exports = LineStop;
```

models/Schedule.js

```
const { DataTypes } = require('sequelize');
const sequelize = require('../config/database');

const Schedule = sequelize.define(
  'Schedule',
  {
    idTime: {
      type: DataTypes.INTEGER,
      primaryKey: true,
      autoIncrement: true,
```

```
    },
    lineStopId: {
      type: DataTypes.INTEGER,
      allowNull: false,
    },
    time: {
      type: DataTypes.TIME,
      allowNull: false,
    },
  },
  {
    tableName: 'horaire',
    timestamps: false,
  }
);

module.exports = Schedule;
```

Relations Sequelize

models/index.js

```
const sequelize = require('../config/database');
const User = require('./User');
const StopCity = require('./StopCity');
const TrainLine = require('./TrainLine');
const LineStop = require('./LineStop');
const Schedule = require('./Schedule');

TrainLine.hasMany(LineStop, {
  foreignKey: 'lineId',
  as: 'stops',
});
LineStop.belongsTo(TrainLine, {
  foreignKey: 'lineId',
  as: 'line',
});

StopCity.hasMany(LineStop, {
  foreignKey: 'cityId',
  as: 'lineStops',
});
LineStop.belongsTo(City, {
  foreignKey: 'cityId',
  as: 'city',
});
```

```
LineStop.hasMany(Schedule, {
  foreignKey: 'lineStopId',
  as: 'schedules',
});
Schedule.belongsTo(LineStop, {
  foreignKey: 'lineStopId',
  as: 'lineStop',
});

module.exports = {
  sequelize,
  User,
  City,
  TrainLine,
  LineStop,
  Schedule,
};
```

middleware/auth.js

```
const jwt = require('jsonwebtoken');
require('dotenv').config();

function auth(requiredRole = null) {
  return (req, res, next) => {
    const authHeader = req.headers.authorization || '';
    const token = authHeader.startsWith('Bearer ')
      ? authHeader.substring(7)
      : null;

    if (!token) {
      return res.status(401).json({ message: 'Token manquant' });
    }

    try {
      const decoded = jwt.verify(token, process.env.JWT_SECRET);
      req.user = decoded;

      if (requiredRole && decoded.role !== requiredRole) {
        return res.status(403).json({ message: 'Accès refusé' });
      }

      next();
    } catch (err) {
      return res.status(401).json({ message: 'Token invalide' });
    }
  };
```

```
}

module.exports = auth;
```

controllers

controllers/authController.js

```
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const { User } = require('../models');
require('dotenv').config();

const SALT_ROUNDS = 10;

exports.register = async (req, res) => {
  try {
    const { username, phone, password } = req.body;

    if (!username || !phone || !password) {
      return res.status(400).json({ message: 'Champs obligatoires manquants' });
    }

    if (password.length < 8) {
      return res
        .status(400)
        .json({ message: 'Le mot de passe doit contenir au moins 8 caractères' });
    }

    const existing = await User.findOne({ where: { phone } });
    if (existing) {
      return res.status(409).json({ message: 'Téléphone déjà utilisé' });
    }

    const passwordHash = await bcrypt.hash(password, SALT_ROUNDS);

    const user = await User.create({
      username,
      phone,
      passwordHash,
      role: 'USER',
    });

    return res.status(201).json({
      idUser: user.idUser,
      username: user.username,
      phone: user.phone,
```

```
      role: user.role,
    });
  } catch (err) {
    console.error('Erreur register:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};

exports.login = async (req, res) => {
  try {
    const { phone, password } = req.body;

    if (!phone || !password) {
      return res.status(400).json({ message: 'Champs obligatoires manquants' });
    }

    const user = await User.findOne({ where: { phone } });
    if (!user) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }

    const isValid = await bcrypt.compare(password, user.passwordHash);
    if (!isValid) {
      return res.status(401).json({ message: 'Identifiants invalides' });
    }

    const token = jwt.sign(
      {
        idUser: user.idUser,
        phone: user.phone,
        role: user.role,
      },
      process.env.JWT_SECRET,
      { expiresIn: process.env.JWT_EXPIRES_IN || '1h' }
    );

    return res.status(200).json({
      token,
      user: {
        idUser: user.idUser,
        username: user.username,
        phone: user.phone,
        role: user.role,
      },
    });
  } catch (err) {
    console.error('Erreur login:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
```

```
  }
};

exports.getMe = async (req, res) => {
  try {
    const user = await User.findByPk(req.user.idUser, {
      attributes: ['idUser', 'username', 'phone', 'role'],
    });

    if (!user) {
      return res.status(404).json({ message: 'Utilisateur introuvable' });
    }

    return res.status(200).json(user);
  } catch (err) {
    console.error('Erreur getMe:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

controllers/searchController.js

```
exports.searchRoute = async (req, res) => {
  try {
    const { from, to } = req.query;

    if (!from || !to) {
      return res.status(400).json({ message: 'Paramètres from et to obligatoires' });
    }

    const lines = await TrainLine.findAll({
      include: [
        {
          model: LineStop,
          as: 'stops',
          include: [
            {
              model: City,
              as: 'city',
              attributes: ['idCity', 'stopCityName'],
            },
            {
              model: Schedule,
              as: 'schedules',
              attributes: ['idSchedule', 'timeStopCity'],
            },
```

```
        ],
      },
    ],
  });

  const results = [];

  lines.forEach((line) => {
    const stops = line.stops.sort((a, b) => a.stopOrder - b.stopOrder);

    const fromStop = stops.find((s) => s.cityId === Number(from));
    const toStop = stops.find((s) => s.cityId === Number(to));

    if (fromStop && toStop && fromStop.stopOrder < toStop.stopOrder) {
      results.push({
        lineId: line.idLine,
        trainLine: line.trainLine,
        from: {
          idCity: fromStop.city.idCity,
          name: fromStop.city.stopCityName,
          schedules: fromStop.schedules,
        },
        to: {
          idCity: toStop.city.idCity,
          name: toStop.city.stopCityName,
          schedules: toStop.schedules,
        },
      });
    }
  });

  return res.status(200).json(results);
 } catch (err) {
  console.error('Erreur searchRoute:', err);
  return res.status(500).json({ message: 'Erreur serveur' });
 }
};
```

controllers/cityController.js

```
const { City } = require('../models');

exports.getAllCities = async (req, res) => {
 try {
  const cities = await City.findAll({
    order: [['stopCityName', 'ASC']],
  });
  return res.status(200).json(cities);
```

```
  } catch (err) {
    console.error('Erreur getAllCities:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

controllers/lineController.js

```
const { TrainLine, LineStop, City, Schedule } = require('../models');

exports.getAllLinesWithStops = async (req, res) => {
  try {
    const lines = await TrainLine.findAll({
      include: [
        {
          model: LineStop,
          as: 'stops',
          include: [
            {
              model: City,
              as: 'city',
              attributes: ['idCity', 'stopCityName'],
            },
            {
              model: Schedule,
              as: 'schedules',
              attributes: ['idSchedule', 'timeStopCity'],
            },
          ],
          order: [['stopOrder', 'ASC']],
        },
      ],
      order: [['trainLine', 'ASC']],
    });

    return res.status(200).json(lines);
  } catch (err) {
    console.error('Erreur getAllLinesWithStops:', err);
    return res.status(500).json({ message: 'Erreur serveur' });
  }
};
```

Routes

routes/auth.routes.js

```
const express = require('express');
const router = express.Router();
const authController = require('../controllers/authController');
const auth = require('../middlewares/auth');

router.post('/register', authController.register);
router.post('/login', authController.login);
router.get('/me', auth(), authController.getMe);

module.exports = router;
```

routes/cityRoutes.js

```
const express = require('express');
const router = express.Router();
const cityController = require('../controllers/cityController');
const auth = require('../middlewares/auth');

router.get('/', auth(), cityController.getAllCities);

module.exports = router;
```

routes/lineRoutes.js

```
const express = require('express');
const router = express.Router();
const lineController = require('../controllers/lineController');
const auth = require('../middlewares/auth');

router.get('/', auth(), lineController.getAllLinesWithStops);

module.exports = router;
```

routes/searchRoutes.js

```
const express = require('express');
const router = express.Router();
const lineController = require('../controllers/lineController');
const auth = require('../middlewares/auth');

router.get('/search', auth(), lineController.searchRoute);

module.exports = router;
```

app.js

```
const app = require('./app');
require('dotenv').config();

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => {
  console.log(`Serveur backend démarré sur le port ${PORT}`);
});
```

🚀 server.js

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const { sequelize } = require('./models');

dotenv.config();

const app = express();

// CORS
const corsOptions = {
  origin: process.env.CORS_ORIGIN || 'http://localhost:3000',
  credentials: true,
};
app.use(cors(corsOptions));

// Body parser
app.use(express.json());

// Routes
const authRoutes = require('./routes/authRoutes');
const cityRoutes = require('./routes/cityRoutes');
const lineRoutes = require('./routes/lineRoutes');
const searchRoutes = require('./routes/searchRoutes');

app.get('/', (req, res) => {
  res.json({ message: 'API Train Routes OK' });
});

app.use('/auth', authRoutes);
app.use('/cities', cityRoutes);
app.use('/lines', lineRoutes);
app.use('/search', searchRoutes);
```

```
// Middleware 404 API
app.use((req, res) => {
  res.status(404).json({ message: 'Route introuvable' });
});

// Synchronisation DB
async function initDatabase() {
  try {
    await sequelize.authenticate();
    console.log('Connexion à la base de données réussie.');
    await sequelize.sync({ alter: false });
  } catch (err) {
    console.error('Erreur de connexion à la base de données:', err);
  }
}

initDatabase();

module.exports = app;
```

3.4 Comptes de test

Créer un admin manuellement :

UPDATE users SET role = 'admin' WHERE username = 'admin';

□ □Endpoints disponibles

Auth

POST /api/auth/register

POST /api/auth/login

GET /api/auth/me

Recherche

GET /api/search?from=Paris&to=Lyon

Villes (admin pour write)

GET /api/cities

POST /api/cities

Mon train

PUT /api/cities/:id

DELETE /api/cities/:id

Lignes (admin pour write)

GET /api/train-lines

POST /api/train-lines

DELETE /api/train-lines/:id

4. Frontend

4.1 Installation

npx create-react-app frontend
cd frontend
npm install react-router-dom axios bootstrap sass i18next react-i18next

4,2 □ □Arborescence

```
src/
│── components/
│   ├── Header.jsx
│   ├── Footer.jsx
│   └── ProtectedRoute.jsx
│
│── context/
│   ├──ThemeContext.jsx
│   └── AuthContext.jsx
│
├── pages/
│   ├── Home.jsx
│   ├── Login.jsx
│   ├── Search.jsx
│   ├── AdminDashboard.jsx
│   └── NotFound .jsx
│
│── services/
│   └── api.js
│
```

```
│── i18n/
│   ├── i18n.js
│   ├── fr.json
│   └── en.json
│
│── styles/
│   └── theme.scss
│
│── App.js
│── index.js
```

4.3 Fichiers

.env.example

```
APP_NAME=My_Train_App
API_URL=127.0.0.1
MONGODB_URI=mongodb://localhost:27017/Api_Train

DB_HOST=localhost
DB_PORT=3306
DB_NAME=train_app
DB_USER=root
DB_PASSWORD=jcg3773

FRONT_URL=http://localhost:5000
```

□ □i18n/i18n.jsx

```
import i18n from 'i18next';
import { initReactI18next } from 'react-i18next';
import fr from './fr.json';
import en from './en.json';

i18n.use(initReactI18next).init({
  resources: { fr: { translation: fr }, en: { translation: en } },
  lng: 'fr',
  fallbackLng: 'en',
  interpolation: { escapeValue: false },
});

export default i18n;
```

fr.json

```
{
  "language": "Langue",
  "theme": "couleur",
  "login": "Se connecter",
  "logout": "Se déconnecter",
  "search": "Rechercher un trajet",
  "from": "Ville de départ",
  "to": "Ville d'arrivée",
  "submit": "Lancer la recherche",
  "allRightsReserved": "Tous droits réservés"
}
```

en.json

```
{
  "language": "Language",
  "theme": "Color",
  "login": "Login",
  "logout": "Logout",
  "search": "Search route",
  "from": "Departure city",
  "to": "Arrival city",
  "submit": "Search",
  "allRightsReserved": "All rights reserved"
}
```

□ □API Axios
services/api.js

```
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:5000/api"
});

api.interceptors.request.use((config) => {

  const token = localStorage.getItem("token");

  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
```

```
  return config;
});

export default api;
```

□ □AuthContext (JWT)
context/AuthContext.jsx

```
import { createContext, useState } from 'react';
import api from '../services/api';

export const AuthContext = createContext();

export function AuthProvider ({ children }) {

  const [user, setUser] = useState(null);

  const login = async (username, phone, password) => {
    const res = await api.post('/auth/login', { username, phone, password });
    localStorage.setItem('token', res.data.token);
    const me = await api.get('/auth/me');
    setUser(me.data);
  };

  const logout = () => {
    localStorage.removeItem('token');
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
};
```

□ □ThemeContext (couleurs demandées)
context/ThemeContext.jsx

```
import { createContext, useState } from 'react';

export const ThemeContext = createContext();

export function ThemeProvider ({ children }) {
```

```
  const [theme, setTheme] = useState('blue');

  const changeTheme = (color) => {
    document.documentElement.setAttribute('data-theme', color);
    setTheme(color);
  };

  return (
    <ThemeContext.Provider value={{ theme, changeTheme }}>
      {children}
    </ThemeContext.Provider>
  );
};
```

□ □SASS thème Bootstrap styles/theme.scss

```
:root[data-theme="blue"] {
  --bs-primary: #0d6efd;
}

:root[data-theme="green"] {
  --bs-primary: #198754;
}

:root[data-theme="red"] {
  --bs-primary: #dc3545;
}

:root[data-theme="yellow"] {
  --bs-primary: #ffc107;
}

:root[data-theme="grey"] {
  --bs-primary: #6c757d;
}
```

□ □ProtectedRoute.jsx

```
import { useContext } from "react";
import { Navigate } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";

export default function ProtectedRoute({ children }) {

  const { user } = useContext(AuthContext);
```

```
  if (!user) return <Navigate to="/login" />;

  return children;
}
```

7. Header dynamique
components/Header.jsx

```
import { useContext } from "react";
import { Link } from "react-router-dom";
import { AuthContext } from "../context/AuthContext";
import { ThemeContext } from "../context/ThemeContext";
import { useTranslation } from "react-i18next";

export default function Header() {

  const { user, logout } = useContext(AuthContext);
  const { changeTheme } = useContext(ThemeContext);
  const { t, i18n } = useTranslation();

  return (

    <nav className="navbar navbar-expand-lg navbar-light bg-light border-bottom">

      <div className="container">

        {/* Application name */}

        <Link className="navbar-brand fw-bold" to="/">
          Train App
        </Link>

        {/* Burger */}

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarContent"
          aria-controls="navbarContent"
          aria-expanded="false"
          aria-label="Toggle navigation"
        >
          <span className="navbar-toggler-icon"></span>
        </button>
```

```
<div className="collapse navbar-collapse" id="navbarContent">

  {/* LEFT MENU */}

  <ul className="navbar-nav me-auto">

    {/* Language */}

    <li className="nav-item dropdown">

      <button
        className="btn nav-link dropdown-toggle"
        data-bs-toggle="dropdown"
      >
        □ □{t("language", "Langue")}
      </button>

      <ul className="dropdown-menu">

        <li>
          <button
            className="dropdown-item"
            onClick={() => i18n.changeLanguage("fr")}
          >
            Français
          </button>
        </li>

        <li>
          <button
            className="dropdown-item"
            onClick={() => i18n.changeLanguage("en")}
          >
            English
          </button>
        </li>

      </ul>
    </li>

    {/* Theme */}

    <li className="nav-item dropdown">

      <button
        className="btn nav-link dropdown-toggle"
        data-bs-toggle="dropdown"
      >
```

```
    □ □t("theme", "Couleur")}
  </button>

  <ul className="dropdown-menu">

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("green")}>
        Vert
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("blue")}>
        Bleu
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("red")}>
        Rouge
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("yellow")}>
        Jaune
      </button>
    </li>

    <li>
      <button className="dropdown-item" onClick={() => changeTheme("grey")}>
        Gris
      </button>
    </li>

  </ul>

 </li>

</ul>

{/* RIGHT MENU */}

<ul className="navbar-nav ms-auto">

 {!user && (

  <li className="nav-item">
```

```
          <Link className="btn btn-primary" to="/login">
            {t("login", "Connexion")}
          </Link>

        </li>

      )}

      {user && (

        <>
          <li className="nav-item">

            <Link className="nav-link" to="/search">
              {t("search", "Recherche")}
            </Link>

          </li>

          <li className="nav-item d-flex align-items-center px-3">

            <span className="small text-muted">
              {user.email}
            </span>

          </li>

          <li className="nav-item">

            <button
              className="btn btn-outline-danger"
              onClick={logout}
            >
              {t("logout", "Déconnexion")}
            </button>

          </li>
        </>
      )}

    </ul>

  </div>

 </div>

</nav>
```

```
  );
}
```

□ □components/Footer.jsx

```
import { useTranslation } from "react-i18next";

export default function Footer() {

  const { t } = useTranslation();

  const year = new Date().getFullYear();

  return (

    <footer
      className="bg-light text-center text-lg-start mt-auto py-3 border-top"
      role="contentinfo"
    >

      <div className="container text-center">

        <p className="mb-1 fw-bold">
          Bus App
        </p>

        <p className="mb-0 small text-muted">
          © {year} Bus App — {t("allRightsReserved", "Tous droits réservés")}
        </p>

      </div>

    </footer>

  );
}
```

□ □Page login
pages/Login.jsx

```
import { useState, useContext } from 'react';
import api from '../services/api';
import { AuthContext } from '../context/AuthContext';

export default function Login() {
  const { login } = useContext(AuthContext);
```

```
  const [form, setForm] = useState({ username: '', password: '' });

  const handleSubmit = async (e) => {
    e.preventDefault();
    const res = await api.post('/auth/login', form);
    login(res.data.token);
  };

  return (
    <div className="container mt-5">
      <h1>Connexion</h1>

      <form onSubmit={handleSubmit} className="col-md-4">
        <label className="form-label">Username</label>
        <input
          className="form-control"
          required
          onChange={e => setForm({ ...form, username: e.target.value })}
        />

        <label className="form-label mt-3">Password</label>
        <input
          type="password"
          className="form-control"
          required
          onChange={e => setForm({ ...form, password: e.target.value })}
        />

        <button className="btn btn-primary mt-3">Login</button>
      </form>
    </div>
  );
}
```

Page recherche trajet
pages/Search.jsx

```
import { useState } from "react";
import api from "../api/axios";

export default function Search() {

  const [from, setFrom] = useState("");
  const [to, setTo] = useState("");
  const [results, setResults] = useState([]);

  const search = async () => {
```

```
    const res = await api.get(`/search?from=${from}&to=${to}`);

    setResults(res.data);
  };

  return (
    <div className="container">

      <h2>Recherche trajet</h2>

      <input
        placeholder="Ville départ"
        onChange={(e) => setFrom(e.target.value)}
      />

      <input
        placeholder="Ville arrivée"
        onChange={(e) => setTo(e.target.value)}
      />

      <button onClick={search}>Search</button>

      <ul>
        {results.map((r, i) => (
          <li key={i}>
            {r.departureCity?.name} → {r.arrivalCity?.name}
          </li>
        ))}
      </ul>

    </div>
  );
}
```

□ □Home + Not found
pages/Home.jsx

```
export default function Home() {
  return (
    <div className="container mt-5">
      <h1>Bus App</h1>
      <p>1. Connectez-vous</p>
      <p>2. Choisissez une ville de départ</p>
      <p>3. Choisissez une ville d'arrivée</p>
      <p>4. Lancez la recherche</p>
```

```
    </div>
  );
}
```

pages/NotFound.jsx

```
export default function NotFound() {
  return (
    <div className="container text-center mt-5">
      <h1>404</h1>
      <p>Page non trouvée</p>
    </div>
  );
}
```

□ □Dashboard admin
pages/AdminDashboard.jsx

```
export default function AdminDashboard() {

  return (
    <div className="container">

      <h2>Admin Dashboard</h2>

      <ul>
        <li><a href="/admin/cities">Cities</a></li>
        <li><a href="/admin/lines">Lines</a></li>
        <li><a href="/admin/schedules">Schedules</a></li>
      </ul>

    </div>
  );
}
```

□ □Routing principal
App.js

```
import { BrowserRouter, Routes, Route } from 'react-router-dom';

import Header from './components/Header';
import Footer from './components/Footer';
```

Mon train

```
import ProtectedRoute from './components/ProtectedRoute';

import Home from './pages/Home';
import Login from './pages/Login';
import Search from './pages/Search';
import AdminDashboard from "./pages/AdminDashboard";
import NotFound from './pages/NotFound';

import { AuthProvider } from "./context/AuthContext";
import { ThemeProvider } from "./context/ThemeContext";

export default function App() {

  return (

    <AuthProvider>
      <ThemeProvider>

        <BrowserRouter>

          <Header />

          <Routes>

            <Route path="/" element={<Home />} />
            <Route path="/login" element={<Login />} />

            <Route
              path="/search"
              element={
                <ProtectedRoute>
                  <Search />
                </ProtectedRoute>
              }
            />

            <Route
              path="/admin"
              element={
                <ProtectedRoute>
                  <AdminDashboard />
                </ProtectedRoute>
              }
            />

            <Route path="*" element={<NotFound />} />

          </Routes>
```

```
        <Footer />

      </BrowserRouter>

    </ThemeProvider>
  </AuthProvider>

  );
}
```

index.js

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App";

import "bootstrap/dist/css/bootstrap.min.css";

import "./i18n/i18n";

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<App />);
```

□ □Lancer le frontend

```
cd frontend
npm install
npm start
```

Application :

http://localhost:3000

5. Readme

```
# Train Routes Finder – Application Web de recherche de trajets en TGV

Application web full-stack permettant de rechercher un trajet en train à grande
vitesse entre deux villes françaises, avec gestion des utilisateurs (rôles
USER / ADMIN), interface accessible (WCAG 2.1), design responsive mobile-first,
multilingue (FR/EN) et multi-thèmes de couleur (vert, bleu, rouge, jaune, gris).



## Fonctionnalités

- Authentification (inscription/connexion) avec rôles USER / ADMIN.
- Recherche d'un trajet entre deux villes (ville de départ, ville d'arrivée) sur
les lignes de train à grande vitesse.
- Visualisation des lignes de train et de leurs arrêts.
- Interface multilingue (français / anglais).
- Thèmes de couleur sélectionnables (vert, bleu, rouge, jaune, gris).
- Respect des bonnes pratiques d'accessibilité (WCAG 2.1 niveau AA) et W3C.
- Tableau de bord administrateur pour gérer les données (villes, lignes, arrêts,
horaires, utilisateurs).



## Technologies



### Frontend

- ReactJS
- React Router
- Bootstrap + Sass (mobile-first, theming via variables)
- i18n (ex : react-i18next)



### Backend

- Node.js
- Express
- Sequelize (ORM)
- MySQL ou MariaDB
- JSON Web Tokens (JWT) pour l'authentification
```

```
### Outils

- Git & GitHub pour le versionning
- Visual Studio Code
- Validateurs W3C + outils d'accessibilité (Lighthouse, extensions navigateur)


## Architecture du projet

```text
.
├── backend/
│   ├── src/
│   │   ├── config/        # Configuration (DB, JWT, etc.)
│   │   ├── models/        # Modèles Sequelize
│   │   ├── controllers/   # Logique métiers
│   │   ├── routes/        # Routes Express (auth, users, trains, admin, etc.)
│   │   ├── middlewares/   # Auth, erreurs, etc.
│   │   ├── app.js
│   │   └── server.js
│   ├── package.json
│   └── .env.example
├── frontend/
│   ├── src/
│   │   ├── components/    # Header, Footer, etc.
│   │   ├── pages/         # Home, Search, CitiesLines, Admin, 404
│   │   ├── context/       # AuthContext, ThemeContext, LanguageContext
│   │   ├── i18n/          # Dictionnaires de traduction FR/EN
│   │   ├── styles/        # Sass + overrides des variables Bootstrap
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── public/            # Favicon, index.html
│   ├── package.json
│   └── .env.example
├── sql/
│   ├── schema.sql         # Script de création de la base
│   └── seed.sql           # Script d'alimentation (jeu d'essai)
└── README.md
```
```

Installation

Prérequis

- Node.js (version LTS recommandée)
- npm ou yarn
- MySQL ou MariaDB
- Git

1. Cloner le dépôt

```
bash
git clone https://github.com/votre-compte/votre-depot.git
cd votre-depot
```

2. Base de données

1. Créer la base et les tables dans MySQL :

   ```
   bash
   mysql -u root -p < sql/train_app_script_import_phpmyadmin.sql
   mysql -u root -p < sql/train_app_script_import_phpmyadmin_seed_test.sql
   ```

2. Adapter si besoin les identifiants dans le fichier `.env` du backend.

3. Backend

```
bash
cd backend
cp .env.example .env
# Éditer .env pour configurer DB_HOST, DB_USER, DB_PASS, DB_NAME, JWT_SECRET,
etc.
npm install
npm run dev   # ou npm start selon votre script

Par défaut l'API écoute sur http://localhost:5000
```

4. Frontend

```
bash
cd ../frontend
cp .env.example .env
# Éditer REACT_APP_API_URL pour pointer vers l'URL de l'API (ex:
http://localhost:5000)
npm install
npm start
```

L'application sera accessible sur `http://localhost:3000`.

Utilisation

- Se connecter avec les comptes fournis dans la documentation (exemples) :
  - Admin : `admin` / `<mot_de_passe_admin>`
  - Utilisateur : `jean` / `<mot_de_passe_user>`
- Une fois connecté :
  - Un utilisateur peut rechercher un trajet, voir les lignes et les villes.
  - Un administrateur peut accéder au tableau de bord et gérer les données.

API (aperçu)

- `POST /auth/register` – inscription.
- `POST /auth/login` – connexion, renvoie un JWT.
- `GET /me` – informations sur l'utilisateur connecté.
- `GET /cities` – liste des villes.
- `GET /lines` – liste des lignes avec leurs arrêts.
- `GET /search?from=<idCity>&to=<idCity>` – recherche de trajet.
- `GET /admin/...` – routes d'administration (protégées, rôle ADMIN).

La documentation détaillée de l'API est disponible via Swagger/OpenAPI sur `/docs` (backend).

Accessibilité & bonnes pratiques

- Respect de WCAG 2.1 niveau AA : alternatives textuelles, contrastes suffisants, navigation clavier, focus visible, etc.[web:4][web:10]
- HTML/CSS valides via les validateurs W3C.
- Design mobile-first, responsive sur différents écrans.